

动漫图形系统综合训练 I

实验指导书 (L1-L14)

从 OpenCV 环境搭建到增量式三维重建：14 次实验 × 软件工程全过程

配套理论课：数字图像基础 (T1-T3) → 视觉特征与匹配 (T4-T8) → 相机几何与重建
(T9-T10)

实验二 图像基本运算与直方图均衡化

软件工程环节：需求确认与范围界定 (需求清单评审、确定 MVP 范围) + 技术调研

(OpenCV 直方图 API) | 对应理论课：T2 数字图像基础 (二) + T3 灰度变换/直方图

灰度变换曲线：线性 / 对数 / 伽马 (gamma)

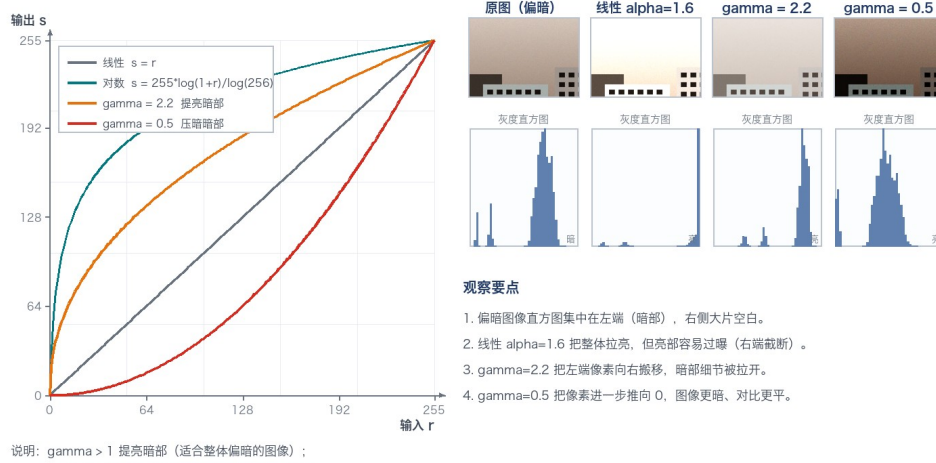


图 1 灰度变换曲线 (线性/对数/伽马)

L2 直方图均衡化前后对比

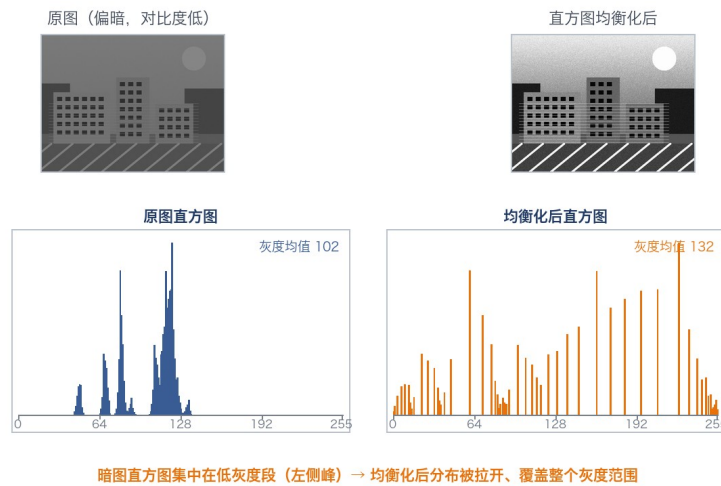


图 2 直方图均衡化前后对比 (实测结果)

0. 学前准备

- 前置知识：复习 T2/T3 中灰度变换、直方图、累积分布函数 (CDF) 的概念。
- 数据准备：准备两张照片：一张整体偏暗（如傍晚拍的），一张亮度正常；把它们放入 data/ 目录。
- 软件准备：确认 matplotlib 可用 (python -c "import matplotlib" 无报错)。
- 复习：能说出直方图的横轴、纵轴各代表什么，CDF 与 PDF 的区别。

1. 实验背景

图像基本运算是所有后续处理的地基：加减乘除、灰度线性变换、伽马校正、直方图统计与直方图均衡化，它们分别解决“亮度调整、对比度增强、分布拉开”等实际问题。直方图描述了图像灰度值的分布，均衡化通过累积分布函数把集中分布的灰度拉开，是最经典的对比度增强手段。

本实验同时进入软件工程的“需求确认”环节：对 L1 撰写的需求清单进行课堂互评，砍掉非核心需求，确定“最小可行产品 (MVP)”范围——例如把“点云可交互显示”保留为 MVP 验收项，把“自动相机参数恢复”推迟到进阶项。

2. 实验目的

1. 掌握图像代数运算（加、减、乘、除、加权混合）与灰度线性/非线性变换。
2. 理解灰度直方图的含义，会用 `cv2.calcHist` 统计并可视化直方图。
3. 掌握直方图均衡化原理，能手写实现并与 `cv2.equalizeHist` 对比。
4. 完成需求清单评审，确定项目 MVP 范围并更新需求文档。

3. 实验要求

1. 任务 1（灰度变换）：实现线性变换 $out = \alpha * img + \beta$ ，观察 $\alpha > 1$ （提高对比度）、 $\beta > 0$ （提亮）的效果；实现伽马校正 $out = 255 * (img / 255)^{(1/\gamma)}$ ，对比 $\gamma = 2.2$ 与 $\gamma = 0.5$ 的效果。
2. 任务 2（代数运算）：对两张图片做加法（`np` 加法与 `cv2.add` 对比，观察溢出差异）、加权混合 `cv2.addWeighted`、减法（可用于两帧求差）。
3. 任务 3（直方图）：用 `cv2.calcHist` 统计灰度图直方图并用 `matplotlib` 绘制；观察暗图、亮图、低对比度图的直方图形状差异。
4. 任务 4（均衡化）：手写均衡化（累积分布函数映射）并调用 `cv2.equalizeHist`，把两张图并排展示原图/均衡化图及各自直方图。
5. 任务 5（软件工程-需求确认）：小组基于 L1 需求文档做课堂互评，按“MVP 必须 / 可推迟 / 删除”三档标注每条需求，输出《需求评审与 MVP 范围》一页文档。
6. 任务 6（软件工程-技术调研）：查 OpenCV 官方文档，记录 `cv2.calcHist`、`cv2.equalizeHist`、`cv2.addWeighted` 的函数签名与关键参数，写入 `docs/api_notes.md`。
7. 加分项：实现自适应伽马校正（按局部亮度自适应调整 γ ），对比固定 γ 的效果。

4. 实验原理

4.1 灰度变换与直方图

灰度变换逐像素建立映射 $s = T(r)$ ，是最简单也最常用的图像增强手段：

$$s = a \cdot r + b$$

其中 a 控制对比度 ($a > 1$ 拉伸对比度)、 b 控制亮度 ($b > 0$ 整体提亮)；伽马校正是最常用的非线性变换：

$$s = 255 \cdot \left(\left(\frac{r}{255} \right) \right)^{1/\gamma}$$

$\gamma > 1$ 时暗部更暗、亮部更亮 (整体变暗)， $\gamma < 1$ 时提亮暗部 (常用于显示设备的校正， $\gamma = 2.2$ 是标准)。直方图 $h(k)$ 统计灰度值为 k 的像素个数，是图像灰度分布的一阶统计；均衡化利用累积分布函数 CDF 做映射：

本实验统一采用公式 $out = 255 \cdot (img/255)^{(1/\gamma)}$ (γ 为伽马值，对应曲线见下图)：

- $\gamma > 1$ (如 2.2)：指数 $1/\gamma < 1$ ，作用是**提亮** (暗部细节变清楚)，这正是显示设备伽马校正的标准用法。
- $\gamma < 1$ (如 0.5)：指数 $1/\gamma > 1$ ，作用是**压暗** (整体变暗、暗部细节丢失)。
- 图 1 图例写成 $\gamma=1/2.2$ 、 $\gamma=1/0.5$ ，标的是“指数”，与代码里的 $1/\gamma$ 是一回事——两处说法不同但指同一个量，按公式写就不会错。

$$s = round\left(\left(255 \cdot CDF(r) \right)\right)$$

使输出直方图近似均匀分布，从而拉开对比度。手写实现时注意：CDF 需先归一化到 $[0, 255]$ ，再对每个灰度值查表映射。

4.2 本实验用到的函数

`cv2.calcHist([img], [0], None, [256], [0, 256])` : 计算单通道直方图 , 返回 (256,1) 数组。

`cv2.equalizeHist(gray)` : 直方图均衡化 , 仅支持单通道 uint8。

`cv2.addWeighted(a, 0.6, b, 0.4, 0)` : 加权相加 $out = 0.6a + 0.4b + 0$ 。

`cv2.add(a, b)` 与 `np` 加法区别 : `cv2.add` 做饱和运算 (溢出截断到 255) , `np.add` 做模 256 运算 (溢出回绕) , 结果差异明显 , 务必对比观察。

5. 实验步骤 (详细)

步骤 1 灰度变换 : 新建 `src/lab2_transform.py` , 读入一张偏暗图片 , 实现线性变换与伽马校正 , 并用 `matplotlib` 画出 1×3 对比子图 (原图/线性增强/伽马校正) 。

```
import cv2, numpy as np
img = cv2.imread("data/photo01.jpg", 0)
linear = np.clip(1.5 * img.astype(np.float32) + 30, 0, 255).astype(np.uint8)
gamma = (255 * ((img / 255.0) ** (1 / 2.2))).astype(np.uint8)
cv2.imwrite("results/l2_linear.jpg", linear)
cv2.imwrite("results/l2_gamma.jpg", gamma)
```

`linear` 图对比度明显提高且整体更亮 ; `gamma` 图暗部细节变清楚、亮部不过曝。

步骤 2 代数运算 : 新建 `src/lab2_arith.py` , 读入两张不同亮度图片 , 对比 `np.add` 与 `cv2.add`、`cv2.addWeighted` 的结果并保存。

```
a = cv2.imread("data/photo01.jpg"); b = cv2.imread("data/photo02.jpg")
add_np = (a.astype(np.int16) + b.astype(np.int16)) % 256
add_cv = cv2.add(a, b)
blend = cv2.addWeighted(a, 0.6, b, 0.4, 0)
```

`add_np` 在亮区会出现回绕产生的“条纹/变色” , `add_cv` 则平滑饱和——这是本实验最重要的对比观察点。

步骤 3 直方图统计 : 用 `calcHist` 统计一张暗图的直方图 , 用 `matplotlib` 绘制柱状图 ; 再对均衡化后的图重新统计 , 观察分布变化。

```
hist = cv2.calcHist([img], [0], None, [256], [0, 256])
import matplotlib.pyplot as plt
plt.plot(hist); plt.title("Histogram"); plt.savefig("results/l2_hist.png")
```

暗图的直方图集中在低灰度段；均衡化后分布被拉开、覆盖更宽范围。

步骤 4 手写均衡化并与 API 对比：按 CDF 公式手写映射，与 cv2.equalizeHist 输出做差值，统计差异像素数。

```
hist = cv2.calcHist([img], [0], None, [256], [0, 256]).ravel()
cdf = hist.cumsum()
cdf = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
manual = cdf[img].astype(np.uint8)
apiv = cv2.equalizeHist(img)
diff = np.abs(manual.astype(int) - apiv.astype(int))
print("最大差异:", diff.max(), " 差异像素占比:", (diff > 1).mean())
```

手写与 API 输出几乎一致（差异像素占比接近 0，微小差异来自取整方式），输出并排对比图 results/l2_eq_compare.jpg。

步骤 5 软件工程-需求评审与 MVP：小组按 3. 节任务 5 输出《需求评审与 MVP 范围》，课堂互评后把评审意见（至少 3 条）写入文档。

文档包含：每条需求的三档标注（MVP/推迟/删除）、MVP 范围一句话总结、至少 3 条评审意见。

步骤 6 技术调研记录：把 calcHist/equalizeHist/addWeighted 的函数签名与参数说明写入 docs/api_notes.md，并注明数据来源（OpenCV 官方文档链接）。

api_notes.md 三个函数签名正确，关键参数（bins、range、alpha 等）解释清楚。

6. 常见问题与排错

问题：直方图画出来是空的 **解决：**calcHist 需要灰度图（单通道）；对彩色图直接调用会报错，先 cvtColor 转灰度。

问题：均衡化后图像发灰/发白 **解决：**均衡化只保证直方图均匀，不保证“好看”；若原图本身对比度正常，均衡化可能过度增强噪声，属于正常现象。

问题：np 加法出现奇怪条纹 解决：这是 uint8 模 256 回绕（溢出）的表现，说明应该用饱和加法 cv2.add 或先转 int16 再截断。

问题：伽马校正图像偏黑 解决：注意公式使用浮点运算：img/255.0 后再乘 255 转回 uint8；直接对 uint8 做除法会丢失精度。

7. 课程思政

科技为民：直方图均衡化是老照片修复、医学影像增强、水下图像清晰化等民生应用的“地基算法”。我国在敦煌壁画数字化、国产纪录片 4K 修复等工程中大量使用这类经典算法，让沉睡的历史档案重新清晰可读——用技术守护文化、服务人民，是计算机人的价值所在。

8. 提交与评分要点

实验报告（变换/运算/直方图/均衡化四部分代码与对比截图、MVP 评审文档要点）、源码 zip、docs/api_notes.md。